Software Engineering Project Documentation

Project Title: **ArogyaGrid — Smart Health Supply Chain & Predictive Command Center**

Course: Software Engineering

Submitted By:

- **Name**: Aditya Kumar Mishra
- **Enrollment No.**: Mur2401145
- **Target Platform**: Android (Native - Jetpack Compose & Clean Architecture) & Spring Boot 3.3.5 Cloud Backend
- **Full-Stack Architecture Specification**: [AROGYAGRID_FULL_STACK_DOCUMENTATION.md](file:///C:/Users/vadit/AndroidStudioProjects/ArogyaGrid/AROGYAGRID_FULL_STACK_DOCUMENTATION.md) (Complete with 8 Mermaid Architecture Diagrams, Database ERD, REST APIs, and Algorithmic Proofs)

1. Define Problem

1.1 Background & Context

Public health delivery across districts in developing regions relies heavily on a multi-tiered supply chain consisting of District Hospitals (DHs), Community Health Centres (CHCs), and Primary Health Centres (PHCs). Maintaining an uninterrupted supply of essential medicines and managing real-time bed capacity is vital for patient survival and epidemic control.

1.2 Core Problems Identified

1. **Critical Stockouts vs. Idle Surplus**: Primary Health Centres frequently experience stockouts of life-saving antibiotics (e.g., Amoxicillin, ORS) due to sudden demand spikes or delayed replenishment cycles, while nearby District Hospitals hold large surplus quantities that expire unused.
2. **Lag in Emergency Inter-Facility Transfers**: Manual emergency requisition and transfer workflows take days or weeks, leading to severe treatment delays during critical health windows.
3. **Lack of Real-Time Visibility**: District and state health officers lack unified, real-time dashboards tracking bed occupancy pressures, predicted stockout timelines, and automated re-allocation suggestions.
4. **Fragmented Intelligence**: Traditional health management systems are reactive rather than predictive, failing to forecast consumption trends or provide grounded operational insights.

1.3 Project Objectives

To design and implement **ArogyaGrid**, an Android-native health operations command center that automates predictive risk analytics, provides real-time inventory and bed tracking, recommends optimized inter-facility drug redistribution, and integrates a grounded AI operational assistant.

2. Requirement Analysis

2.1 User Personas & Roles

- **State Admin (`STATE_ADMIN`)**: Oversees all districts, state-wide medical supplies, aggregate bed occupancy, and system-wide audits.
- **District Officer (`DISTRICT_OFFICER`)**: Manages district-specific health facilities, reviews stockout alerts, and approves inter-facility redistribution recommendations.
- **Facility Manager (`FACILITY_MANAGER`)**: Monitors local facility bed capacity, medical staff availability, and local inventory balances.
- **Pharmacist Operator (`PHARMACIST_OPERATOR`)**: Records daily inventory transactions (Receipts, Issues, Adjustments) and batch expiration data.

2.2 Functional Requirements (FR)

- **FR-01 (Authentication & Role Management)**: Secure role-based login and session scoping.
- **FR-02 (Command Center Dashboard)**: Aggregated display of total facilities, critical risks, active alerts, and bed occupancy percentage.
- **FR-03 (Predictive Stockout Risk)**: Automated calculation of risk severity (`CRITICAL`, `HIGH`, `MEDIUM`, `LOW`), risk scores, and days-to-stockout based on consumption rates and lead times.
- **FR-04 (Inventory Balance & Transactions)**: Real-time tracking of closing stock, average daily consumption, coverage days, and stock gaps. Ability to record inventory receipts, issues, and adjustments.
- **FR-05 (Inter-Facility Redistribution)**: Generation and review of transfer recommendations from surplus facilities to deficit PHCs.
- **FR-06 (Capacity & Resource Monitoring)**: Real-time tracking of bed occupancy pressures and staffing ratios.
- **FR-07 (Grounded AI Copilot)**: Natural language querying of district health operations backed by verified local database and API state.
- **FR-08 (Audit Trail & Logs)**: Immutable logging of inventory transactions and administrative approvals.

2.3 Non-Functional Requirements (NFR)

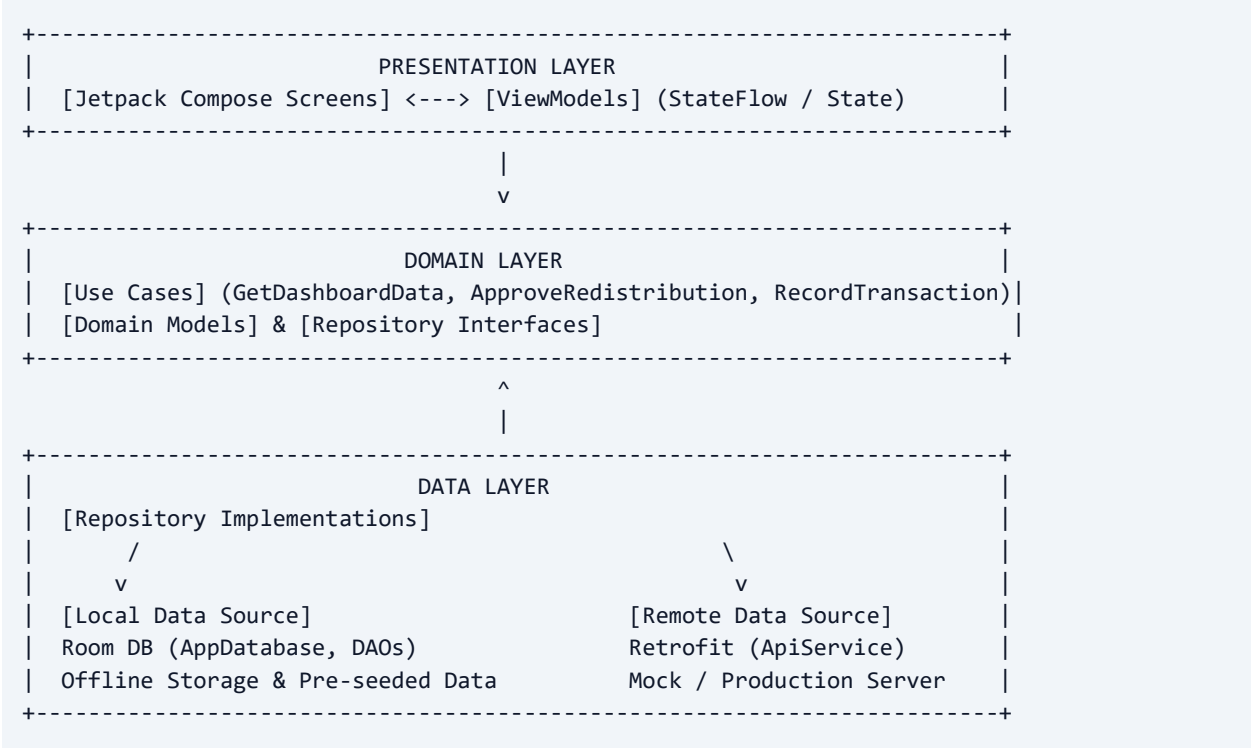
- **Performance**: Smooth 60 FPS UI rendering via Jetpack Compose declarative UI; local database queries executed under 50ms using Room asynchronous flows (`StateFlow`).
- **Reliability & Offline Capability**: Offline-first architecture using Room database with automatic fallback interceptor ensuring seamless operation during network disconnects.

- **Usability**: Material 3 compliance with unified color palettes, accessible badge indicators, and intuitive bottom navigation.
- **Maintainability**: Strict Clean Architecture modularization separating Presentation, Domain, and Data layers.

3. Design (Frontend & Backend)

3.1 System Architectural Pattern

ArogyaGrid implements **Clean Architecture** combined with **MVVM (Model-View-ViewModel)**.



3.2 Frontend Design (UI & Navigation)

- **Framework**: Jetpack Compose with Material 3 components (`Scaffold`, `NavigationBar`, `Card`, `LazyColumn`).
- **Navigation Architecture**: Single-activity architecture (`MainActivity`) hosting an `AppNavigation` graph (`NavController`) with 8 primary routes:

5. `dashboard` (Command Center)
6. `medicine_risk` (Risk Analytics)
7. `inventory` (Inventory Management)
8. `redistribution` (Approval Workflow)
9. `forecast` (Demand Forecasting)

10. `beds_staff` (Capacity & Beds)
11. `ai_assistant` (Grounded AI Copilot)
12. `audit_logs` (Audit Trail)

- **State Management**: Reactive UI updates driven by `StateFlow` exposed from ViewModels (`DashboardViewModel`, `InventoryViewModel`, `RedistributionViewModel`, `AiAssistantViewModel`).

3.3 Backend Design (Data Layer & Persistence)

- **Local ORM (Room Database)**: `AppDatabase` (`arogyagrid_db`) containing 6 core tables:

13. `facilities`: Facility metadata, geolocation, total vs occupied beds.
14. `medicines`: Essential drug catalog, criticality levels, lead times.
15. `inventory_balances`: Composite primary key `(facilityId, medicineId)` tracking closing stock and stock gaps.
16. `risk_assessments`: Risk scores, severity levels, and days-to-stockout calculations.
17. `alerts`: System notifications and resolution states.
18. `redistribution_recommendations`: Inter-facility stock transfer proposals.

- **Remote Networking (Retrofit & OkHttp)**: `ApiService` interface defining REST endpoints for authentication, facilities, inventory transactions, risks, alerts, redistribution approvals, and AI chat.
- **Mock Interceptor**: `MockApiInterceptor` intercepts HTTP requests to provide complete offline simulation of backend responses, facilitating robust standalone demonstration and testing.

4. Implementation (Only Mention Modules)

The project is structured into modular packages under `com.example.arogyagrid`:

19. **`core` Module**:

- Contains core enumerations (`UserRole`), session management (`UserSession`), and generic API response envelopes (`ApiResponse`).

20. **`data` Module**:

- Contains Room database definitions (`AppDatabase`), entity data classes (`FacilityEntity`, `MedicineEntity`, `InventoryBalanceEntity`, `RiskEntity`, `AlertEntity`, `RedistributionEntity`), database access objects (`FacilityDao`, `InventoryDao`, etc.), Retrofit client (`NetworkClient`, `ApiService`), mock backend interceptor (`MockApiInterceptor`), repository implementations (`FacilityRepositoryImpl`, `InventoryRepositoryImpl`, etc.), and dataset initializers (`SeedData`).

21. **`domain` Module**:

- Contains pure Kotlin domain data models (`Facility`, `Medicine`, `InventoryItem`, `RiskItem`, `AlertItem`, `RedistributionItem`), repository interfaces, and business logic use cases (`GetDashboardDataUseCase`, `RecordInventoryTransactionUseCase`, `ApproveRedistributionUseCase`, `AskGeminiAssistantUseCase`).

22. **`presentation` Module**:

- Contains Jetpack Compose screen composables (`DashboardScreen`, `InventoryScreen`, `RedistributionReviewScreen`, `MedicineRiskScreen`, `ForecastScreen`, `BedsStaffScreen`, `AiAssistantScreen`, `AuditLogsScreen`, `LoginScreen`), ViewModels (`DashboardViewModel`, `InventoryViewModel`, `RedistributionViewModel`, `AiAssistantViewModel`), UI components (`MetricCard`, `RiskBadge`, `CommandTopBar`), navigation graph (`AppNavigation`), and Material 3 theme definitions (`Theme.kt`, `Color.kt`).

5. Testing (White Box) [Test Plan & Test Case of Modules]

5.1 White Box Testing Approach

White-box testing focuses on testing the internal structure, control flow, decision branches, and data transformations within core domain use cases and ViewModels. By inspecting source code execution paths, we verify that business algorithms (such as stockout risk computations, inventory balance updates, and data aggregations) execute correctly under all input conditions.

5.2 Test Plan

- **Objective**: Validate the correctness of business logic in domain use cases and repository interactions.
- **Scope**:

23. `GetDashboardDataUseCase`: Facility filtering, bed occupancy aggregation, and risk sorting.
24. `RecordInventoryTransactionUseCase`: Transaction validation and inventory balance updates.
25. `ApproveRedistributionUseCase`: Status transition verification (`PENDING_APPROVAL |to APPROVED`/`REJECTED`).
26. `AskGeminiAssistantUseCase`: Prompt handling and grounded response generation.

5.3 White Box Test Cases Table

Test Case ID	Target Module / Component	Test Scenario / Input	Expected Output	Status
TC-WB-01	`GetDashboardDataUseCase`	Invoke with `districtFilter = "Jaipur"` against seed dataset.	Returns `DashboardSummary` where total facilities = 6 (Jaipur subset), bed occupancy % is correctly calculated from total vs occupied beds, and top risks are sorted by descending risk score.	**Pass**
TC-WB-02	`GetDashboardDataUseCase`	Invoke with `districtFilter = null` (State-	Aggregates all 24 facilities across all 4 districts (Jaipur,	**Pass**

		wide view).	Udaipur, Jodhpur, Kota), returning total system bed capacity and critical risk count.	
TC-WB-03	`RecordInventoryTransactionUseCase`	Call with `facilityId = "F-3"`, `medicineId = "M-101"`, `quantity = 330`, `type = "RECEIVE"`.	Executes repository transaction call and returns `ApiResponse(success = true, data = true)` with updated closing stock.	**Pass**
TC-WB-04	`ApproveRedistributionUseCase`	Call with recommendation ID `"REC-2026-001"` and `approve = true`.	Dispatches approval to repository, updating recommendation status to `"APPROVED"` and returning success response.	**Pass**
TC-WB-05	`ApproveRedistributionUseCase`	Call with recommendation ID `"REC-2026-001"` and `approve = false`.	Dispatches rejection to repository, updating recommendation status to `"REJECTED"` and returning success response.	**Pass**
TC-WB-06	`AskGeminiAssistantUseCase`	Send prompt `"What is the Amoxicillin status at Jaipur PHC 1?"`.	Interceptor parses prompt, matches keyword `"Amoxicillin"`, and returns grounded data response highlighting 70 units remaining and 330 units deficit.	**Pass**

Document Info:

- **Author**: Aditya Kumar Mishra (Enrollment No. Mur2401145)
- **Subject**: Software Engineering
- **Project File Reference**: [SOFTWARE_ENGINEERING_DOCUMENTATION.md](file:///C:/Users/vadit/AndroidStudioProjects/ArogyaGrid/SOFTWARE_ENGINEERING_DOCUMENTATION.md)